جامعة الشهيد حمه لخضر – الوادي
قسم الإعلام الآلي

تقرير العمل التطبيقي

مقياس أنماط التصميم بلغة Java

المشروع: نظام إدارة حسابات الطلبة

العمل الأول – UniAccounts

إعداد الطالبتين:

بن علي الأميرة جوهر

حامدي آمنة

إشراف الأستاذ: يعقوب

السنة الجامعية: 2026 / 2027

1. المقدمة وأهداف المشروع

1. المقدمة

في إطار الأعمال التطبيقية لمقياس أنماط التصميم بلغة Java، أنجزنا الجزء الأول من مشروع UniAccounts، وهو نظام مصغر يهدف إلى إدارة حسابات الطلبة داخل الجامعة.

تتمثل فكرة المشروع في إنشاء حسابات للطلبة تحتوي على معلوماتهم الشخصية والأكاديمية، ثم تخزين هذه الحسابات في مستودع مركزي، مع تسجيل العمليات التي تتم داخل النظام في سجل التدقيق Audit Log.

يركز هذا العمل على تطبيق ثلاثة أنماط تصميم إنشائية تساعد على تنظيم الكود وتحسين طريقة إنشاء الكائنات وإعادة استخدامها، وهي: Builder وSingleton وPrototype.

2. أهداف المشروع

- إنشاء حسابات للطلبة تحتوي على معلومات شخصية وأكاديمية.
- التحقق من صحة المعلومات قبل إنشاء الحساب.
- تسهيل إنشاء الحسابات التي تضم حقولاً إجبارية واختيارية.
- ضمان وجود مستودع مركزي واحد لتخزين الحسابات.
- ضمان وجود سجل تدقيق واحد لتسجيل العمليات.
- إنشاء حسابات جديدة انطلاقاً من نموذج حساب موجود مسبقاً.
- تنظيم البرنامج لتسهيل تطويره وصيانته مستقبلاً.

3. البنية العامة للنظام

3.1. الصنف StudentAccount

يمثل هذا الصنف حساب الطالب، ويحتوي على ثمانية حقول: رقم التسجيل id، الاسم الكامل fullName، البريد الإلكتروني email، الهاتف phone، المستوى level، التخصص major، المنحة scholarship، وقائمة المقررات courses.

يتحقق النظام من أن رقم التسجيل والاسم غير فارغين، وأن البريد الإلكتروني يحتوي على الرمز @، وأن المستوى واحد من L1 أو L2 أو L3 أو M1 أو M2. المستوى الافتراضي هو L1، والمنحة false، وقائمة المقررات فارغة افتراضياً.

3.2. الصنف AccountRepository

يتولى إدارة حسابات الطلبة وتخزينها في مستودع مركزي باستعمال LinkedHashMap. يوفر عمليات إضافة الحساب، والبحث عنه برقم التسجيل، واسترجاع جميع الحسابات، وحساب عددها. كما يمنع إضافة رقم تسجيل مكرر.

3.3. الصنف AuditLogger

يسجل العمليات التي تتم داخل النظام مع التاريخ والوقت. وتتم إضافة كل عملية إلى سجل مركزي، كما يمكن استرجاع قائمة العمليات المسجلة.

3.4. الصنف Main

يمثل نقطة انطلاق التطبيق، ويستعمل لإنشاء الحسابات وإضافتها إلى المستودع واختبار البحث ومنع تكرار رقم التسجيل والتحقق من Singleton وعرض سجل التدقيق.

4. أنماط التصميم المستعملة

4.1. نمط Builder – نمط البناء

المشكلة: يحتوي حساب الطالب على ثمانية حقول، بعضها إجباري وبعضها اختياري. استعمال منشئ واحد طويل يجعل الكود صعب القراءة وقد يؤدي إلى أخطاء في ترتيب المعاملات.

الحل: أنشأنا الصنف الداخلي Builder داخل StudentAccount. يتم تحديد الحقول الإجبارية أولاً، وهي رقم التسجيل والاسم والبريد الإلكتروني، ثم تضاف الحقول الاختيارية مثل الهاتف والمستوى والتخصص والمنحة والمقررات. تستعمل الدالة build() لإنشاء الحساب.

الفائدة: يجعل إنشاء الحساب مرناً وواضحاً، ويسمح بتحديد الحقول الاختيارية حسب الحاجة، مع التحقق من البيانات قبل إنشاء الكائن. وبدونه سنحتاج إلى منشئ طويل يصعب استعماله وصيانته.

4.2. نمط Singleton – نمط النسخة الوحيدة

المشكلة: يحتاج النظام إلى مستودع واحد لجميع الحسابات وسجل تدقيق واحد لجميع العمليات. إنشاء نسخ متعددة قد يؤدي إلى تشتت البيانات.

الحل: طبقنا Singleton في AccountRepository وAuditLogger. جعلنا المنشئ private، ووفرنا الدالة getInstance() للوصول إلى النسخة الوحيدة. ويستعمل AccountRepository آلية Holder لإنشاء النسخة عند الحاجة بطريقة آمنة مع تعدد الخيوط.

الفائدة: توحيد تخزين الحسابات وتسجيل العمليات وتجنب إنشاء نسخ متعددة. وبدون هذا النمط قد يحتفظ كل جزء من البرنامج بمستودع أو سجل منفصل.

5. نمط Prototype والاختبارات

4.3. نمط Prototype – نمط النموذج الأولي

المشكلة: قد ينتمي عدد كبير من الطلبة إلى التخصص والمستوى نفسيهما ويتابعون المقررات نفسها. إعادة إدخال المعلومات الأكاديمية المشتركة لكل طالب تؤدي إلى تكرار العمل.

الحل: طبقنا Prototype في StudentAccount باستعمال Cloneable وإعادة تعريف clone(). كما أنشأنا copyWithIdentity() لنسخ نموذج حساب مع تغيير رقم التسجيل والاسم والبريد الإلكتروني، مع الاحتفاظ بالمعلومات الأكاديمية.

الفائدة: إعادة استعمال المعلومات المشتركة وتقليل تكرار الكود وتسهيل إنشاء حسابات متشابهة. وبدون هذا النمط يجب إنشاء كل حساب من البداية وإدخال معلوماته كاملة.

5. الاختبارات والنتائج

بعد تطبيق الأنماط الثلاثة، شغلنا البرنامج من خلال Main وأجرينا اختبارات للتأكد من صحة الوظائف.

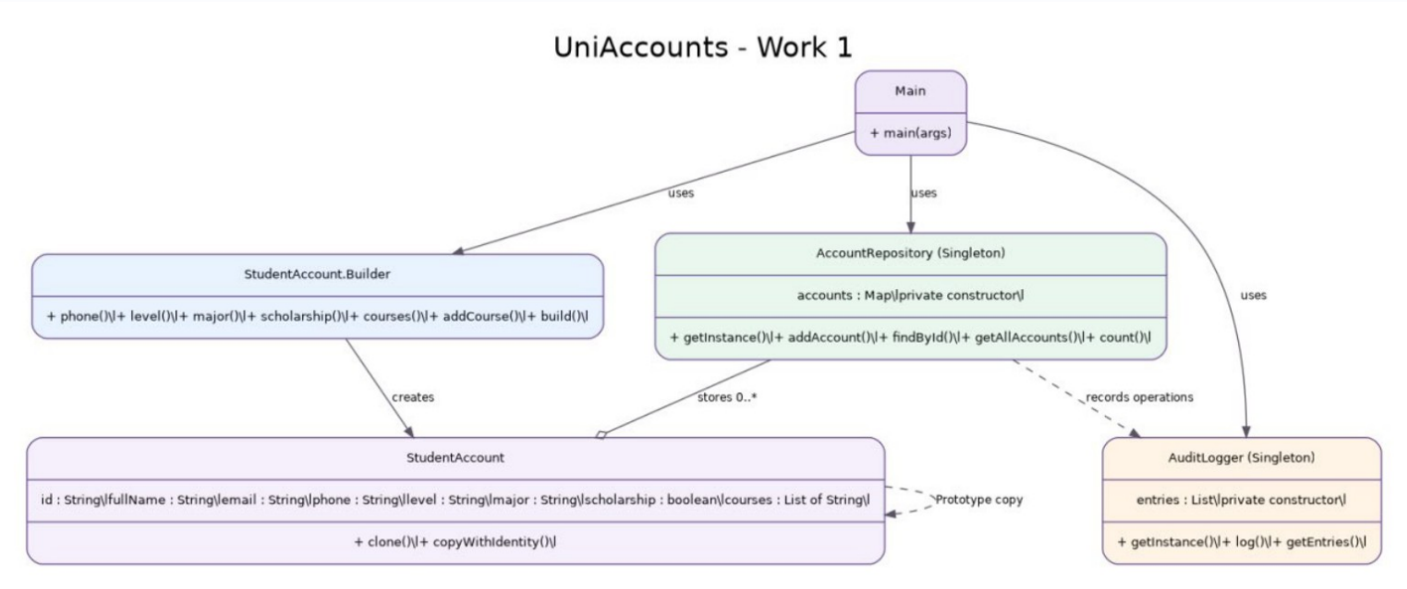
- إنشاء ثلاثة حسابات للطلبة: Amina Benali وYoussef Ali وSara Ouali.
- إضافة الحسابات إلى المستودع المركزي وعرض العدد الإجمالي، وكانت النتيجة ثلاثة حسابات.
- البحث عن حساب باستعمال رقم التسجيل، ونجح الاختبار.
- محاولة إضافة حساب برقم تسجيل مكرر، فرفض النظام العملية وأظهر رسالة خطأ.
- اختبار Singleton، وأكدت النتيجة أن استدعاء getInstance() يعيد النسخة نفسها للمستودع ولسجل التدقيق.
- عرض العمليات المسجلة في Audit Log مع التاريخ والوقت.

6. الخاتمة

أنجزنا في هذا العمل النسخة الأولى من نظام UniAccounts باستعمال Java وثلاثة أنماط إنشائية. ساعد Builder على إنشاء الحسابات بمرونة، وضمن Singleton وجود مستودع وسجل تدقيق موحدين، بينما سمح Prototype بإنشاء حسابات جديدة انطلاقاً من نموذج موجود. وتمثل هذه النسخة قاعدة العمل الثاني الذي سيضيف أنماطاً وخصائص جديدة.

7. الملاحق: صور المشروع

الشكل 1: مخطط الأصناف UML للعمل الأول



الشكل 2: نتيجة تشغيل البرنامج في نافذة Terminal

```
Terminal  Local ×  +  ⌄                                                    AI Agents ⌄  ⋮  —
javac -d out $(find src -name "*.java")
java -cp out uniaccounts.Main
========== UniAccounts | Work 1 ==========
[AUDIT] [2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260001 (Amina Benali)
[AUDIT] [2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260002 (Youssef Ali)
[AUDIT] [2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260003 (Sara Ouali)

--- Accounts in central repository ---
StudentAccount{id='20260001', fullName='Amina Benali', email='amina@univ.dz', level='L3', major='Informatique', scholarship=false, courses=[POO, Systemes, Res
eaux]}
StudentAccount{id='20260002', fullName='Youssef Ali', email='youssef@univ.dz', level='L3', major='Informatique', scholarship=false, courses=[POO, Systemes, Re
seaux]}
StudentAccount{id='20260003', fullName='Sara Ouali', email='sara@univ.dz', level='M1', major='Informatique', scholarship=true, courses=[Architecture, Bases de
 donnees]}

Total accounts: 3

--- Search test ---
StudentAccount{id='20260002', fullName='Youssef Ali', email='youssef@univ.dz', level='L3', major='Informatique', scholarship=false, courses=[POO, Systemes, Re
seaux]}

--- Duplicate ID validation test ---
Expected validation: An account with id 20260001 already exists.

--- Singleton identity test ---
Same repository instance? true
Same audit logger instance? true

--- Audit log entries ---
[2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260001 (Amina Benali)
[2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260002 (Youssef Ali)
[2026-11-10 19:35:25] ACCOUNT_CREATED - Stored account 20260003 (Sara Ouali)
```